

Constructing the Modern QLSA T-Count

A Step-by-Step Explanation of Table V

HHL paper revision: task 04

20 September 2026

This note explains the implemented function `paper_table_v.py:f(p)`, based on Table V (PDF page 22) of Dalzell et al., arXiv:2211.12489v1 [1]. It follows the discrete-adiabatic QLSA used inside that paper’s quantum interior-point method. It does not use the earlier Shortcut cost construction.

Result for the running example. For $(N, \kappa, \epsilon, \delta) = (1024, 100, 0.01, 0.01)$, our stated parameter choices give

$$T_{\text{attempt}} \approx 1.2386477814166392 \times 10^{16}, \quad f(p) = 86,705,344,699,164,751.$$

The first quantity prices one circuit attempt; the second budgets seven attempts. These are evaluations of the paper’s resource *model*, not certified finite circuit upper bounds. In particular, the source uses approximations for the adiabatic schedule and rotation synthesis. Rounding a model value to an integer does not remove those approximations.

Step 1. Define the problem and what is being counted

The task is to prepare the normalized quantum solution of $Ax = b$:

$$|x\rangle = \frac{A^{-1}b}{\|A^{-1}b\|}, \quad p = (N, \kappa, \epsilon, \delta).$$

Here N is the original matrix dimension, κ bounds its ordinary spectral condition number, ϵ is the solution-state error target used in the paper’s error model, and δ is our allowed abort probability. The state-error convention follows the source’s vector-norm formulation, rather than a complete scientific-output error budget.

The T count includes T and $T^\dagger$ gates in matrix access, RHS preparation, adiabatic evolution, and filtering. The default state-only mode excludes tomography, interior-point iterations, classical preprocessing, and quantum error correction. Step 10 adds optional full-vector tomography. QLSA is only one subroutine of QIPM; we do not multiply by the number of optimization iterations.

The calculation proceeds in this order:

$$\boxed{N, \kappa \longrightarrow L, K \longrightarrow Q, d \longrightarrow \text{gate precisions} \longrightarrow T_{\text{attempt}}}$$

$$\boxed{\delta \longrightarrow \text{retry cap} \longrightarrow f(p)}.$$

Step 2. Convert the input to the block-encoding normalization

Pad to $L = 2^\ell$, where $\ell = \lceil \log_2 N \rceil$. Positive nonsingular padding within the original singular-value range preserves the condition bound; zero-padding the matrix would introduce unwanted singularity.

The paper’s block encoding contains $A/\|A\|_F$ as a block of a larger unitary. Therefore its effective inverse gap is

$$K \geq \kappa_F(A) := \|A\|_F \|A^{-1}\|_2 = \frac{\|A\|_F}{\sigma_{\min}(A)}. \quad (1)$$

The ordinary condition number is instead $\|A\|_2/\sigma_{\min}(A)$. Thus, writing $\rho = \|A\|_F/\|A\|_2$, we need $K \geq \kappa\rho$. The default adapter uses $\rho \leq \sqrt{L}$ and sets $K = \kappa\sqrt{L}$. This is a conservative bound, not a claim that every matrix attains it.

For our example,

$$L = 1024, \quad \ell = 10, \quad \sqrt{L} = 32, \quad K = 100 \times 32 = 3200.$$

A certified tighter ratio can be passed as `frobenius_over_spectral`; a direct K can be passed as `effective_inverse_gap`. Using $K = 100$ would describe a different normalization assumption from our default example.

Step 3. Determine the adiabatic schedule and filter degree

The solver first uses discrete adiabatic evolution to obtain a constant-accuracy state, then QSVT eigenstate filtering to improve its accuracy. This is the solver of Costa et al. [2], as adapted in the QIPM paper’s Proposition 2.

Adiabatic parameter. The source gives $Q = 2CK + O(\sqrt{K})$, and its numerical calculation uses $Q \approx 2CK$ (Eq. 66). We implement

$$Q = \lceil 2CK \rceil, \quad C = 2000 \quad \text{by default.} \quad (2)$$

The value $C = 2000$ is the paper’s numerical assumption for general matrices, not its proven analytic constant. The omitted remainder is not bounded by this estimator.

Filter degree. With polynomial-error allowance ϵ_{qsp} , Eq. (67) gives $d = 2K \ln(2/\epsilon_{\text{qsp}})$. Our adapter rounds upward to an even integer:

$$d = 2 \lceil K \ln(2/\epsilon_{\text{qsp}}) \rceil. \quad (3)$$

Step 4 sets $\epsilon_{\text{qsp}} = \epsilon/5 = 0.002$, giving

$Q = 12,800,000, \quad d = 44,210.$

Q is a schedule parameter, not the total matrix-query count. There are $Q + d$ forward and $Q + d$ inverse matrix calls per attempt.

Step 4. Allocate the error before pricing the gates

More accurate synthesized gates cost more T gates, so the total accuracy target must first be converted into per-operation tolerances. Following the error accounting displayed in the source’s Eq. (64), we use

$$\epsilon_{\text{QLSP}} \lesssim \epsilon_{\text{qsp}} + 2(Q + d)\epsilon_G + 4(Q + d)\epsilon_h + 4Q\epsilon_{\text{ar}} + 2d\epsilon_z. \quad (4)$$

Here the approximate relation emphasizes our use of the source’s resource/error model; it is not a fresh certification of the implemented circuit and its postselection.

Tolerance	What it measures
ϵ_{qsp}	Approximation error of the eigenstate-filtering polynomial.
ϵ_G	Error in one matrix block-encoding unitary.
ϵ_h	Error in one RHS state-preparation unitary.
ϵ_{ar}	Error in each synthesized adiabatic interpolation rotation.
ϵ_z	Error in each synthesized QSVT phase rotation.

The coefficients count repeated uses, or give the source’s error allowance for those uses. Matrix and RHS errors must consequently be much smaller than the final state error.

Our explicit adapter choice: split ϵ equally among the five contributions. For $s = \epsilon/5$, set

$$\epsilon_{\text{qsp}} = s, \quad \epsilon_G = \frac{s}{2(Q + d)}, \quad \epsilon_h = \frac{s}{4(Q + d)}, \quad (5)$$

$$\epsilon_{\text{ar}} = \frac{s}{4Q}, \quad \epsilon_z = \frac{s}{2d}. \quad (6)$$

First choose ϵ_{qsp} , then compute d , then the remaining tolerances; there is no circular dependency. This equal allocation is our choice, not the paper’s tomography-dominated 90% allocation. Eq. (65) uses a different coefficient for ϵ_z ; we retain the larger $2d$ coefficient in Eq. (64).

For the running example:

Tolerance	Value
ϵ_{qsp}	0.002
ϵ_G	$7.7856092356 \times 10^{-11}$
ϵ_h	$3.8928046178 \times 10^{-11}$
ϵ_{ar}	3.90625×10^{-11}
ϵ_z	$2.2619316897 \times 10^{-8}$

Substitution into the right side of Eq. (4) gives five contributions of 0.002, summing to 0.01.

Step 5. Price one matrix block encoding and one RHS preparation

A matrix block encoding is a unitary U_A with an encoded block A/α , where here $\alpha = \|A\|_F$. The source constructs it from two data-dependent state-preparation operations, with coherent data loading and uncomputation. It imports the circuit costs from Clader et al. [3] and selects the *minimum-depth* dense-data construction. It does not assume a free unit-cost QRAM query, and it does not exploit matrix sparsity.

For $L = 2^\ell$, Table III supplies

$$T_{Cbe} = [12\log_2(1/\epsilon_G) + 56]L^2 - 24L - 12\log_2(1/\epsilon_G) - 32\ell - 32, \quad (7)$$

$$T_{Cbe} = T_{Cbe} + 16(L - 1). \quad (8)$$

Subscript C means T *count*, “be” means block encoding, and “cbe” adds a quantum control. The $16(L - 1)$ term is the source’s incremental control cost. The dominant L^2 term reflects this particular general dense-data circuit; it is not a universal lower bound for matrix access.

Table IV supplies the RHS preparation cost

$$T_{Csp} = [12\log_2(1/\epsilon_h) + 40]L - 12\log_2(1/\epsilon_h) - 16\ell - 40. \quad (9)$$

It prices the unitary preparing $|b\rangle$ from a basis state; its inverse has the same cost. The source assigns the same T count to its controlled version. These data-loading rotations are already priced inside Eqs. (7)–(9); do not charge them again as solver-level rotations.

Both tables suppress some doubly/triply logarithmic synthesis terms. Their constants are imported circuit-accounting results, not fitted by our program. The source’s rotation-synthesis model is $R_T(\zeta) = 3\log_2(1/\zeta)$; actual compiled gate lists are not provided by this estimator.

Numerically,

$$\begin{aligned} T_{Cbe} &\approx 481,234,131.9233, \\ 16(L - 1) &= 16,368, \\ \boxed{T_{Cbe}} &\approx 481,250,499.9233, \\ T_{Csp} &\approx 465,268.9794. \end{aligned}$$

These noninteger values are resource-model evaluations. The final function rounds the complete per-attempt total; it does not assert that a fractional number of physical gates can be executed.

Step 6. Multiply primitive costs by their use counts

Table V combines the data-access primitives with the remaining adiabatic and filtering gates. The bookkeeping is:

Operation	Multiplicity	Contribution to T count
Controlled matrix BE and inverse	$2(Q + d)$	$2(Q + d)T_{C_{cbe}}$
RHS preparation and inverse	$4(Q + d)$	$4(Q + d)T_{C_{sp}}$
Adiabatic axial rotations	$4Q$	$4Q[3 \log_2(1/\epsilon_{ar})]$
Other adiabatic gates	Q groups	$Q(24\ell + 31)$
QSVT phase rotations	d	$d[3 \log_2(1/\epsilon_z)]$
Other filter gates	d groups	$d(32\ell - 2)$

The two oracle multiplicities include adjoints. There are $2Q$ adiabatic interpolation gates, each decomposed into two axial rotations, explaining $4Q$ and hence the factor $12Q$ in the rotation cost. The QSVT phase count is d ; the $2d$ in the error allocation is the larger source error-accounting coefficient, not twice as many priced phase gates.

The terms $24\ell + 31$ and $32\ell - 2$ are the source’s residual T costs after expanding its circuit decompositions (Sec. IV.F and Figs. 1–6). They account for the remaining controlled logic/reflections. We retain these aggregate gate counts rather than inventing a new Toffoli decomposition or replacing them with unspecified overhead.

Adding the six contributions gives the QLSS column of Table V:

$$\begin{aligned}
 T_{\text{attempt}} = & 2(Q + d)T_{C_{cbe}} + 4(Q + d)T_{C_{sp}} \\
 & + 12Q \log_2(1/\epsilon_{ar}) + Q(24\ell + 31) \\
 & + 3d \log_2(1/\epsilon_z) + d(32\ell - 2).
 \end{aligned} \tag{10}$$

Thus the matrix-access contribution has exactly the form

$$\underbrace{\text{number of matrix calls}}_{2(Q+d)} \times \underbrace{\text{T count per matrix call}}_{T_{C_{cbe}}}.$$

It is only one term of the full solver cost. In particular, $Q T_{C_{cbe}}$ alone would omit forward/inverse multiplicity, filtering, RHS access, and other gates.

The other column. “Controlled QLSS” prices the source’s tomography sign-recovery circuit. Its extra T cost relative to Eq. (10) is

$$20Q + 3d \log_2(1/\epsilon_z) + 12(L - 1) \log_2(1/\epsilon_{tsp}) + 16(L - \ell - 1).$$

It is not automatically added to ordinary state preparation. Our low-level `table_v()` exposes it when `epsilon_tsp` is supplied.

Step 7. Evaluate the single-attempt cost

For $Q = 12,800,000$ and $d = 44,210$,

$$n_{\text{BE}} = 2(Q + d) = 25,688,420, \quad n_{\text{RHS}} = 4(Q + d) = 51,376,840.$$

Using the per-call values from Step 5 gives the following exclusive contributions, rounded here for display:

Contribution	T count
Controlled matrix access	$1.2362564967240332 \times 10^{16}$
RHS preparation/unpreparation	$2.3904049913518 \times 10^{13}$
Adiabatic rotations	5.3107852430×10^9
Other adiabatic gates	3,468,800,000
QSVT phase rotations	3.3685194183×10^6
Other filter gates	14,058,780
Total	$1.2386477814166392 \times 10^{16}$

The unrounded total begins 12,386,477,814,166,392.186487.... Matrix block encoding accounts for approximately 99.8% of it. Its size is driven by the dense L^2 access circuit, tight per-call precision, and large adiabatic schedule parameter. It does not imply that every modern QLSA implementation has this cost.

Step 8. Apply a retry cap for the requested failure allowance

Table V prices one attempt. The ideal algorithm's stated success probability is at least $1/2$. Assuming independent attempts with that success lower bound, r consecutive failures have probability at most 2^{-r} . Our adapter therefore chooses

$$r = \lceil \log_2(1/\delta) \rceil, \quad \boxed{f(p) = r \lceil T_{\text{attempt}} \rceil}. \quad (11)$$

For $\delta = 0.01$, $r = 7$ and $2^{-7} = 0.0078125 < 0.01$. Hence

$$\begin{aligned} \lceil T_{\text{attempt}} \rceil &= 12,386,477,814,166,393, \\ f(p) &= 7 \times 12,386,477,814,166,393 \\ &= \boxed{86,705,344,699,164,751}. \end{aligned}$$

Execution may stop on the first success. The expected number of attempts is at most two under the same ideal success model, whereas seven is a cap. The source's tomography-failure parameter is a different quantity. Success after approximate circuit synthesis has not been recertified here, so this retry calculation remains conditional on the stated success assumption.

Step 9. Reproduce the calculation and interpret it correctly

The implementation uses Python 3.11 or later and only the standard library. In the `table-v` directory:

```
from paper_table_v import f, estimate
p = {"N": 1024, "kappa": 100,
     "epsilon_state": "0.01", "delta": "0.01"}
assert f(p) == 86705344699164751
result = estimate(p)
print(result["T_per_attempt_ceiling"])
print(result["matrix_calls"])
print(result["T_controlled_BE"])
```

Run `python3.11 reproduce.py` to regenerate the JSON results and validation. The estimator uses 80-digit decimal arithmetic. The rounded example agrees at 40 and 100 digits; the tests also check both Table V columns against an independent small hand calculation and verify error allocation, query multiplicities, and retry boundaries.

Ingredient	Status in this estimator
Table III/IV/V algebra	Implemented as displayed in the source.
Frobenius ratio $\rho = \sqrt{L}$	Our generic upper-bound choice; replaceable by certified matrix information.
$C = 2000$, $Q \approx 2CK$	Paper's numerical assumption/approximation; a schedule remainder is omitted.
Five equal error contributions	Our state-only allocation, using Eq. (64).
Even-degree and final integer rounding	Our reporting/implementation conventions.
Synthesis and access costs	Source approximations; some subleading terms omitted.
Seven attempts	Our cap, conditional on success probability at least $1/2$.

What this result establishes. It makes the chosen published resource model evaluable as a function of problem parameters, with an auditable decomposition. It does not prove a minimum cost or provide a fully certified sufficient T budget. Full certification would require a finite valid adiabatic schedule, verified gate synthesis, and a success/error analysis for the resulting approximate circuit. This distinction also prevents a direct claim of algorithmic superiority from comparison with the much looser earlier Shortcut construction.

Relation to original HHL. The original HHL phase-estimation algorithm requires controlled Hamiltonian simulation. A controlled block encoding is not itself time evolution; Table III's per-call cost cannot simply replace a Hamiltonian-simulation cost. This note specifically prices the discrete-adiabatic solver.

Step 10. Optionally include full-vector readout

The estimator now offers two separate output tasks. With `include_readout=False` (the default), Steps 1–9 and their numerical result are unchanged. With `include_readout=True`, it uses the paper’s real-vector tomography protocol (Sec. IV.D, Proposition 4). This returns a classical approximation of the *normalized* solution vector, including relative signs. It does not recover its physical norm or implement the PDE’s scalar observable.

Let $\xi = \text{epsilon_readout}$ be the total normalized-vector error target, and let $\delta_{\text{ro}} = \text{delta_readout}$. They default to the input state-error and failure targets, but have distinct meanings. Our explicit allocation is

$$\begin{aligned}\epsilon_{\text{tomo}} &= 0.9\xi, \\ \epsilon_{\text{core}} &= \min\{\epsilon_{\text{state}}, 0.05\xi/1.58\}, \\ \epsilon_{\text{tsp}} &= 0.05\xi/(1.58\sqrt{L}).\end{aligned}$$

The solver schedule/precisions are recomputed using ϵ_{core} . These choices ensure the source’s modeled output-error expression

$$\epsilon_{\text{tomo}} + 1.58\sqrt{L}\epsilon_{\text{tsp}} + 1.58\epsilon_{\text{core}} \leq \xi.$$

The program also checks the applicability condition $\epsilon_{\text{tomo}} + \sqrt{2L}\epsilon_{\text{tsp}} + \sqrt{2}\epsilon_{\text{core}} \leq 1/2$. The 90/5/5 split is our adapter choice, not an additional theorem from the paper.

The required repetitions *of each circuit family* are

$$k_{\text{tomo}} = \left\lceil \frac{57.5L \ln(6L/\delta_{\text{ro}})}{\epsilon_{\text{tomo}}^2(1 - \epsilon_{\text{tomo}}^2/4)} \right\rceil.$$

Use k_{tomo} ordinary QLSS circuits for magnitudes and k_{tomo} controlled sign-recovery circuits (the right column of Table V). With both costs evaluated at the tightened precisions,

$$T_{\text{with readout}} = k_{\text{tomo}}(\lceil T_{\text{QLSS}} \rceil + \lceil T_{\text{controlled QLSS}} \rceil).$$

The sample formula counts the unsuccessful solver flags already: there is no extra seven-attempt or $1/p$ multiplier, nor a separately added state-only delivery. An exclusive breakdown is one solver execution plus $(k_{\text{tomo}} - 1)$ further ordinary executions and k_{tomo} sign-recovery executions.

For $N = 1024$, $\kappa = 100$, $\epsilon_{\text{state}} = \xi = 0.01$, and $\delta_{\text{ro}} = 0.01$,

$$k_{\text{tomo}} = 9\,688\,792\,223, \quad T_{\text{with readout}} = 271\,767\,176\,507\,565\,562\,710\,978\,575.$$

This large total describes full-vector tomography, not the cost of a single scalar readout. It inherits the source model’s success and synthesis assumptions. Classical reconstruction, norm recovery, and QEC remain excluded.

```
f(p) # unchanged state-only mode
f(dict(p, include_readout=True,
      epsilon_readout="0.01", delta_readout="0.01"))
```

The returned `readout` object records the retuned schedule, error budget, both circuit prices, and their exclusive contributions. Top-level `Q`, `d`, and `T_retry_budget` retain the state-only baseline for comparison; `T_total` and `f(p)` select the requested workload.

References

- [1] A. M. Dalzell et al., *End-to-end resource analysis for quantum interior point methods and portfolio optimization*, arXiv:2211.12489v1 (2022). Tables III–IV, p. 14; Table V, p. 22; Eqs. (64)–(67), p. 26. <https://arxiv.org/pdf/2211.12489v1>.
- [2] P. C. S. Costa et al., *Optimal Scaling Quantum Linear-Systems Solver via Discrete Adiabatic Theorem*, PRX Quantum **3**, 040303 (2022). <https://doi.org/10.1103/PRXQuantum.3.040303>.
- [3] B. D. Clader et al., *Quantum Resources Required to Block-Encode a Matrix of Classical Data*, IEEE Transactions on Quantum Engineering **3**, 3103323 (2022). <https://doi.org/10.1109/TQE.2022.3231194>.